

D.Y. Patil Academic Education Excellence Federation's
Dr. D. Y. Patil Technical Campus

Polytechnic | Engineering | MCA

(Approved by AICTE, Recognized by DTE, Govt. of Maharashtra, Affiliated to Savitribai Phule Pune University, Pune)
Sr.No.32/1/A/7, Near Talegaon Railway Station, A/P Varale, Talegaon Dabhade, Tal-Maval, Dist-Pune 410 507

Website: www.dypatilcs.com Email: principal@dypatilcs.com

AICTE ID: 1- 43358498993 I SPPU PUN CODE: CMMP021930 | ISHE CODE: C -71504 | DTE CODE: 6991

Dr. D. Y. Patil
Founder Dr. D. Y. Patil Group

Dr. Sushant V. Patil
Chairman

Adv. Anuja Sushant Patil
Director

Dr. Suresh Shirbahadurkar
Principal

Dr. D. Y. Patil Technical Campus
(Affiliated to Savitribai Phule Pune University, Pune)
DEPARTMENT OF ELECTRONICS AND COMPUTER ENGINEERING

A REPORT ON
“DIFFERENT SOFTWARE TOOLS USED IN THE
ELECTRONICS & COMPUTER ENGINEERING INDUSTRY”

Submitted By
Aditya Suresh More
Branch: Electronics and Computer Engineering, Division: B
Roll No.: 44
University: Savitribai Phule Pune University (SPPU)
College: Dr. D. Y. Patil Technical Campus

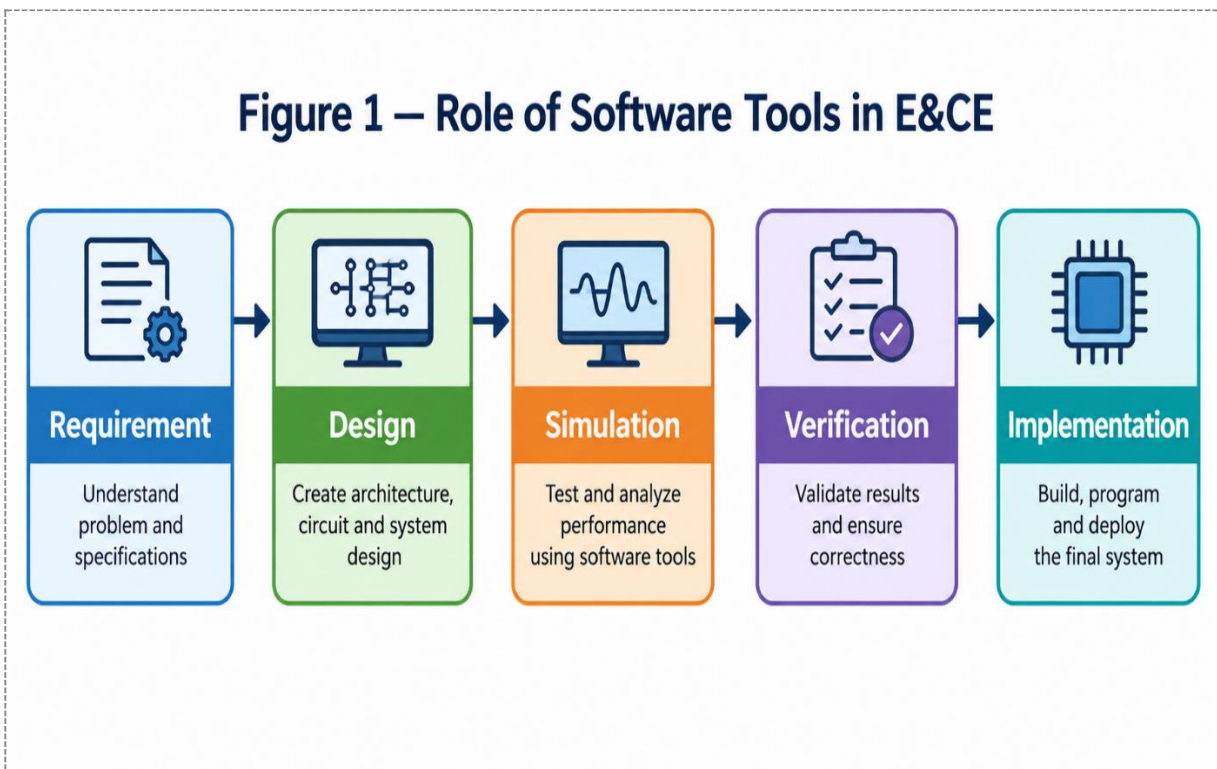
1. Introduction

Electronics and Computer Engineering (E&CE) integrates electronic hardware design with computing and embedded intelligence. As modern systems grow more complex, software tools have become essential for design, simulation, verification, and implementation, reducing cost, time, and risk compared to purely manual prototyping.

This report studies the role of such tools in E&CE, focusing on three widely used examples from different domains: NI Multisim (circuit simulation), Proteus Design Suite (embedded/microcontroller simulation), and Xilinx/AMD Vivado (FPGA and digital hardware design).

Objectives of the Report

- To understand the role of software tools in Electronics and Computer Engineering.
- To study commonly used engineering software across different design domains.
- To understand the purpose, features, and working of Multisim, Proteus, and Vivado.
- To describe their basic design and simulation workflows.
- To present simple, beginner-level demonstrations for each tool.
- To compare the three tools on relevant technical parameters.
- To relate these tools to industry relevance and student career development.



[FIGURE 1 — Role of Software Tools in Electronics & Computer Engineering]
Figure 1: Conceptual role of software tools in the E&CE design cycle

2. Software Tools Used in Electronics & Computer Engineering

Different engineering tasks call for different categories of software, depending on the design domain and intended outcome. Common categories in E&CE are listed below.

Circuit Simulation

- NI Multisim
- LTspice
- PSpice

Embedded-System and Microcontroller Simulation

- Proteus Design Suite
- Other microcontroller-specific simulators

PCB Design

- KiCad
- Altium Designer
- OrCAD

FPGA and Digital Design

- Xilinx/AMD Vivado
- Intel Quartus Prime

Mathematical and Engineering Analysis

- MATLAB / Simulink

VLSI and IC-Level Design

- MicroWind
- Other layout-oriented tools

Tool choice depends on the design requirement, domain, and target platform — a simulated circuit, an embedded system, or programmable hardware.

Software Category	Example Tool	Main Purpose
Circuit Simulation	Multisim	Electronic circuit analysis
Embedded Simulation	Proteus	Microcontroller/circuit simulation
FPGA Design	Vivado	FPGA and digital design
PCB Design	KiCad / Altium	PCB development
Mathematical Simulation	MATLAB / Simulink	Modelling and analysis
VLSI / IC Design	MicroWind	IC-related design studies

3. NI Multisim

What is Multisim?

NI Multisim is a schematic-capture and circuit-simulation environment for designing, analysing, and testing analog, digital, and mixed-signal circuits before physical construction.

Why was Multisim developed?

It was developed to give students and engineers a virtual laboratory to study circuit behaviour without the cost and risk of building a physical circuit for every iteration.

Developer / Background

Multisim originated from Electronics Workbench and is now developed by National Instruments (NI); it is widely used in academic electronics laboratories.

Who Uses Multisim?

- Electronics students
- Engineering educators
- Circuit designers
- Electronics engineers
- Researchers
- Training laboratories

Major Features

- Schematic-based circuit design
- Extensive component libraries
- Interactive circuit simulation (SPICE-based)
- Virtual measurement instruments (multimeter, oscilloscope, function generator)
- Waveform and frequency-response analysis
- Educational analysis tools for teaching circuit theory

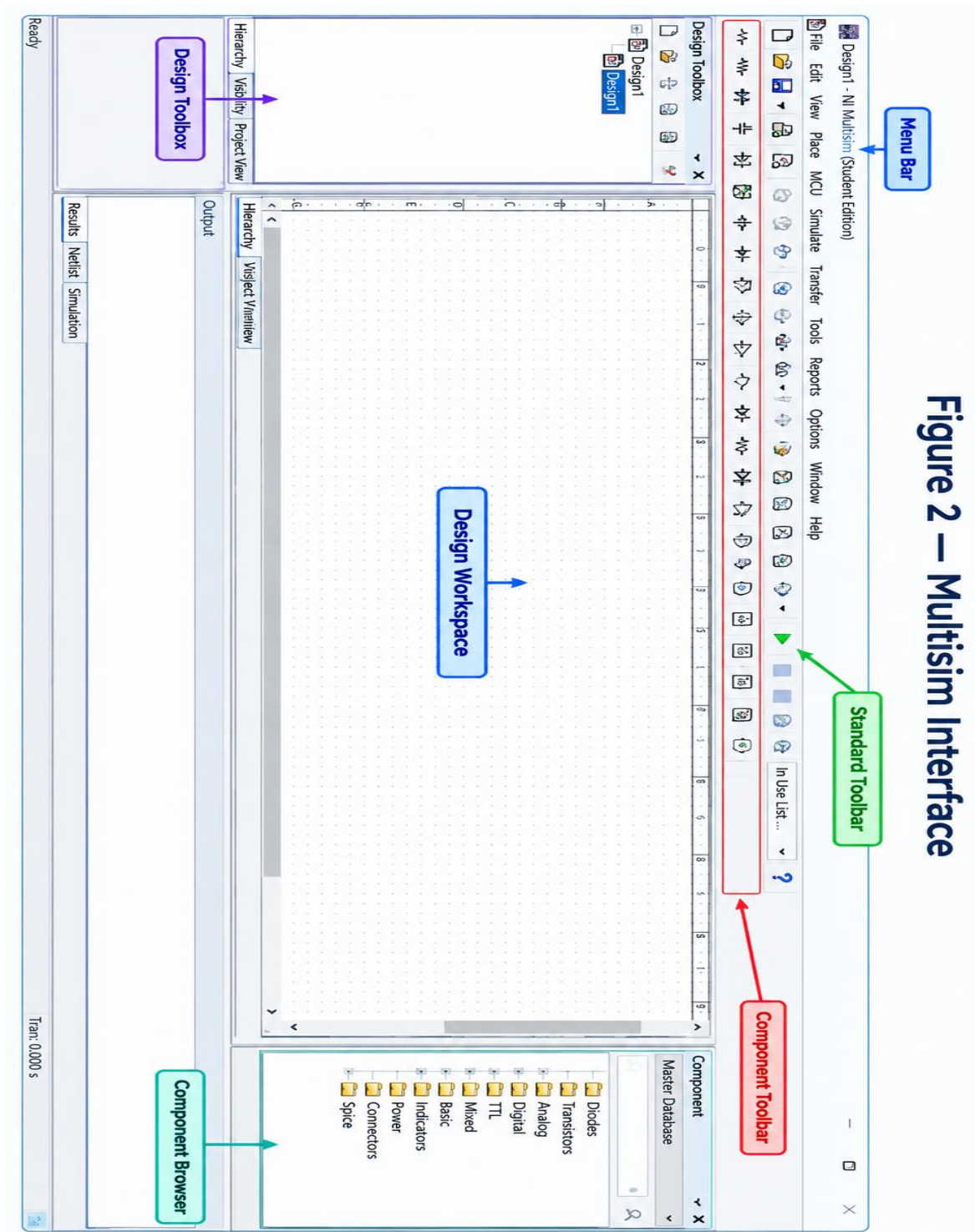


Figure 2: Typical Multisim circuit-design and simulation environment

3.1 Working Principle and Basic Workflow

1. Create or open a schematic.
2. Select the required components from the library.
3. Place components on the schematic sheet.
4. Connect the components to form the circuit.
5. Set appropriate component values/parameters.
6. Add measurement instruments to the circuit.
7. Run the simulation.
8. Observe the output on the connected instruments.
9. Analyse the simulated results.
10. Compare the simulated result with theoretical calculations.

This can be summarised as a simple design flow: Circuit Design → Component Setup → Simulation → Measurement → Analysis → Verification.

How Students Can Use Multisim

- Verify theoretical circuit calculations practically.
- Experiment safely with component values without hardware risk.
- Visualise waveforms and signal behaviour interactively.

How Engineers Can Use Multisim

- Prototype and validate circuit ideas before hardware fabrication.
- Perform preliminary analysis of analog and digital sub-circuits.
- Support documentation and design-review processes.

Advantages

- Safe, low-cost experimentation.
- Immediate visual feedback through virtual instruments.
- Useful for both academic teaching and preliminary industrial testing.

Limitations

- Simulation models may not capture every real-world non-ideality.
- Complex or very high-frequency circuits may require more specialised tools.

3.2 Practical Demonstration Using Multisim

Demonstration: Simulation of a Basic RLC / Rectifier Circuit (select the circuit relevant to your coursework).

Aim

To simulate a simple electronic circuit in Multisim and observe its voltage/current response using virtual measurement instruments.

Required Components

Component	Typical Role
Resistor	Current/voltage control
Capacitor	Energy storage / filtering
Inductor or Diode	Depends on selected circuit type
Voltage Source	Circuit excitation
Measurement Instrument	Result observation (multimeter/oscilloscope)

Procedure

11. Open a new Multisim design.
12. Place the required components from the library.
13. Connect the components as per the chosen circuit topology.
14. Assign realistic example values to each component.
15. Attach a multimeter/oscilloscope for measurement.
16. Run the simulation.
17. Observe the resulting voltage, current, or waveform.
18. Interpret the result and relate it to theoretical expectations.

Expected Observation

The instrument should show a response consistent with theory — e.g. a charging/discharging curve, a rectified waveform, or a steady-state value.

Result

The demonstration shows how Multisim verifies circuit behaviour before physical implementation.

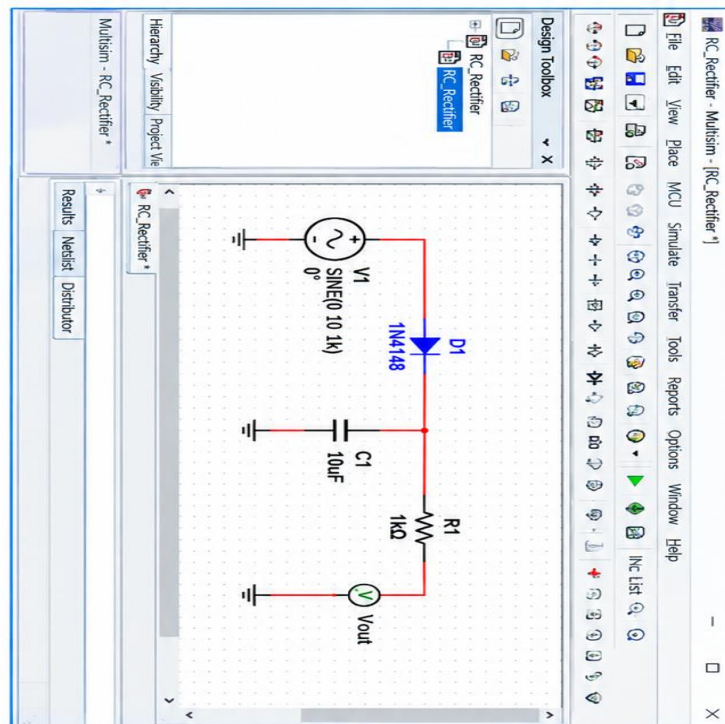


Figure 3 — Multisim Circuit Screenshot

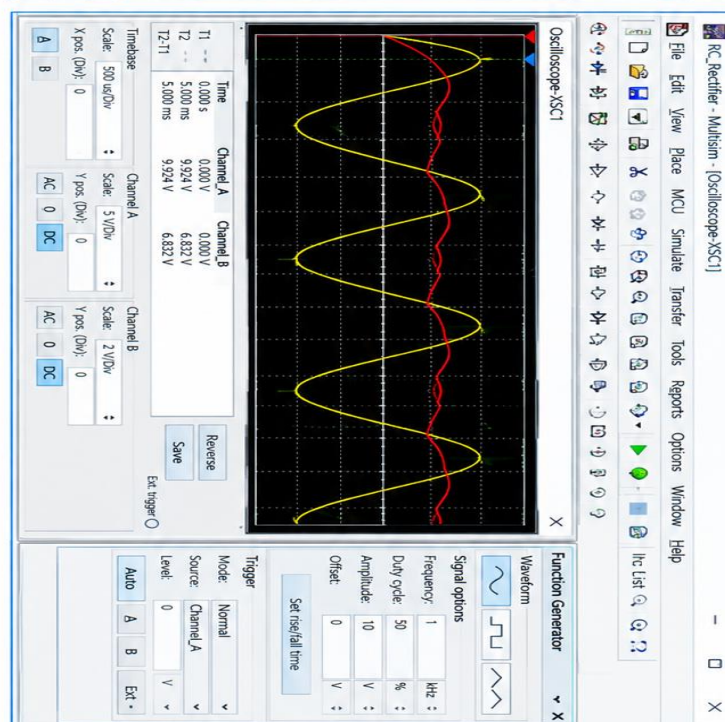


Figure 4 — Multisim Waveform

4. Proteus Design Suite

What is Proteus?

Proteus Design Suite combines schematic circuit simulation with microcontroller and embedded-system simulation, allowing an entire embedded circuit — hardware and firmware — to be tested virtually.

Why was Proteus developed?

It lets engineers and students validate embedded hardware/firmware interaction without needing a physical microcontroller board at every development stage.

Developer / Company Background

Proteus is developed by Labcenter Electronics, specialising in electronic design automation (EDA) and simulation software.

Who Uses Proteus?

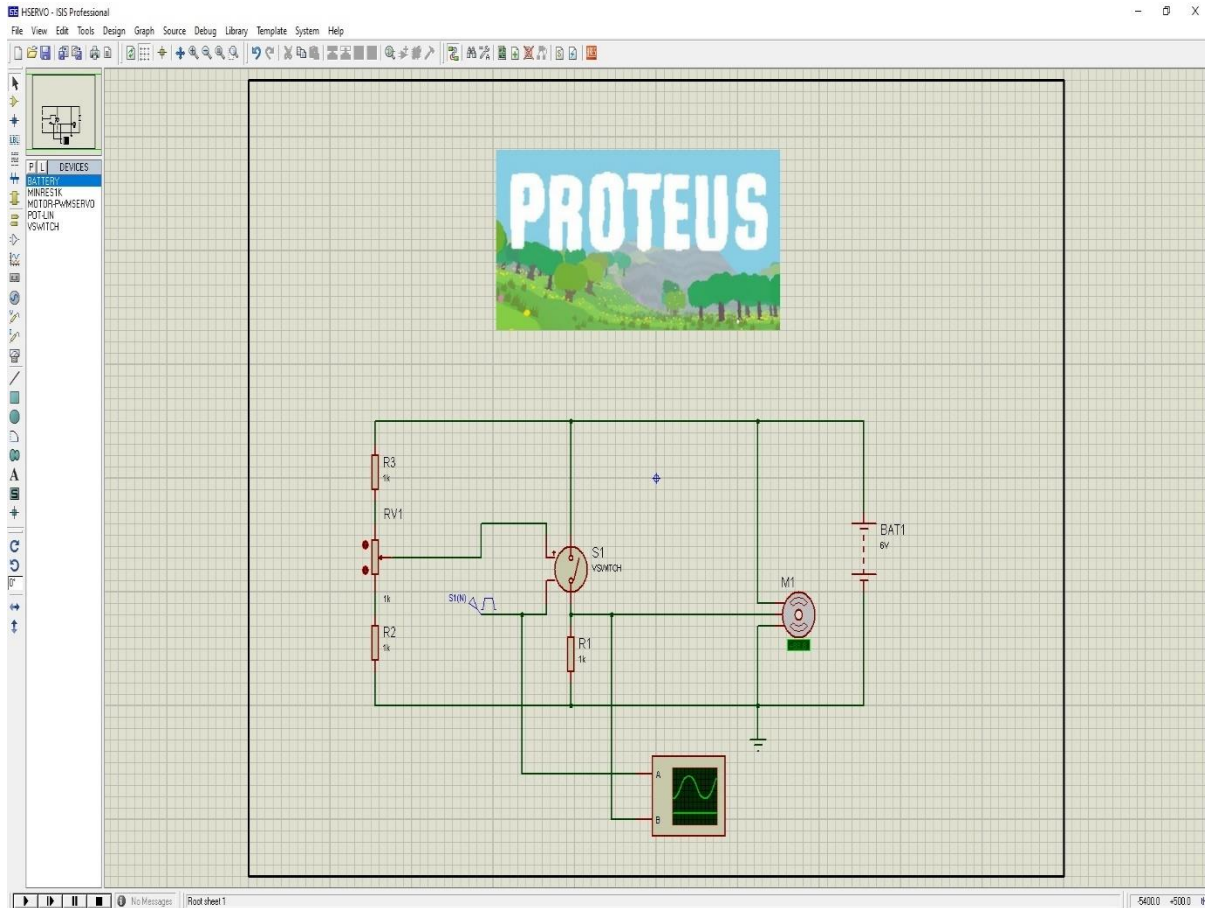
- Electronics students
- Embedded-system learners
- Electronics engineers
- Microcontroller developers
- PCB designers
- Educational laboratories

Major Features

- Circuit simulation similar to Multisim
- Microcontroller and embedded-system simulation
- Virtual instruments for measurement
- Schematic design tools
- PCB-related design workflow support

Applications

- LED and display control circuits
- Sensor-interfacing circuits
- Basic motor-control concepts
- Digital logic circuits
- Embedded-system prototypes



4.1 Working Principle and Basic Workflow

Workflow: Schematic → Component Selection → Connections → Program/Configuration → Simulation → Observation → Debugging. This lets both circuit and firmware logic be validated together before physical hardware construction.

Microcontroller Simulation

- Selecting an appropriate microcontroller model.
- Connecting the microcontroller to supporting circuit elements.
- Preparing the program/firmware logic.
- Loading or configuring the program in the simulation.
- Running the simulation.
- Observing the resulting behaviour of the circuit.

Applications

- LED control
- Seven-segment displays

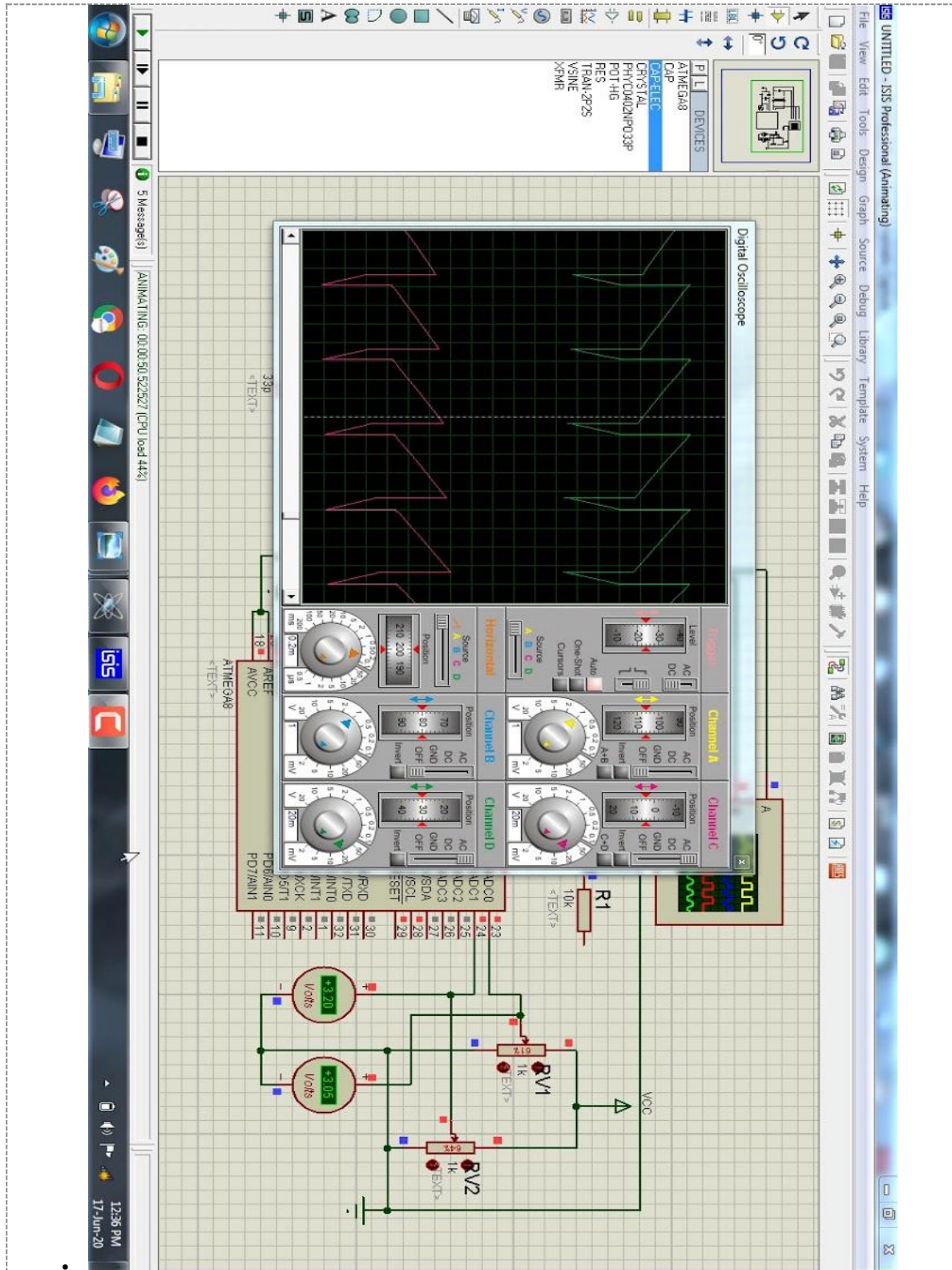
- Sensor interfacing
- Motor-control concepts
- Digital logic circuits
- Embedded-system prototypes
- Basic communication concepts

Advantages

- Allows combined hardware-and-firmware testing.
- Reduces dependency on physical microcontroller boards during early development.
- Helps visualise embedded-system behaviour interactively.

Limitations

- Simulation cannot fully replicate all real-world electrical and timing behaviour.
- Certain peripherals or hardware-specific features may not be completely modelled.
- Final validation on physical hardware remains necessary.



- [FIGURE 6 — Proteus Microcontroller Simulation — INSERT IMAGE HERE]
- Figure 6: Microcontroller-based circuit simulated in Proteus

4.2 Practical Demonstration Using Proteus

Demonstration: Microcontroller-Based LED Control.

Aim

To simulate a simple microcontroller circuit that controls an LED and observe the expected output behaviour.

Required Elements

Element	Role
Microcontroller	Executes the control logic
LED with resistor	Output indicator
Power supply	Circuit excitation
Program/firmware logic	Defines LED switching behaviour

Circuit Setup

The microcontroller drives an LED through a current-limiting resistor via its output pin.

Program / Logic

The firmware switches the LED on and off at defined intervals, illustrating basic digital output control.

Procedure

19. Create a new Proteus project.
20. Select the required components.
21. Place the microcontroller on the schematic.
22. Connect the LED and supporting circuit elements.
23. Prepare the program logic.
24. Configure/load the program in the simulation environment.
25. Start the simulation.
26. Observe the LED output.

27. Verify that the observed behaviour matches the intended logic.

5. Xilinx Vivado / AMD Vivado

What is Vivado?

Vivado is a design environment for developing digital hardware on Field-Programmable Gate Arrays (FPGAs), supporting design entry, simulation, synthesis, implementation, and hardware programming.

Why was Vivado developed?

It provides one integrated environment for increasingly complex FPGA designs, replacing older, fragmented digital-design toolchains.

Developer / Company Background

Vivado was originally developed by Xilinx; following AMD's acquisition of Xilinx, it is now also associated with AMD branding while serving the same core purpose.

Who Uses Vivado?

- FPGA engineers
- Digital-design engineers
- Hardware designers
- Verification engineers
- Engineering students
- Researchers

Major Concepts

- FPGA — Field-Programmable Gate Array, reconfigurable digital hardware.
- HDL — Hardware Description Language, used to describe digital logic.
- RTL — Register-Transfer Level, an abstraction describing data flow between registers.
- Simulation — verifying logical correctness before hardware mapping.
- Synthesis — converting RTL into a hardware-oriented representation.
- Implementation — mapping and routing the design onto FPGA resources.

- Constraints — timing and pin-assignment requirements for the design.
- Bitstream — the configuration file used to program the FPGA.

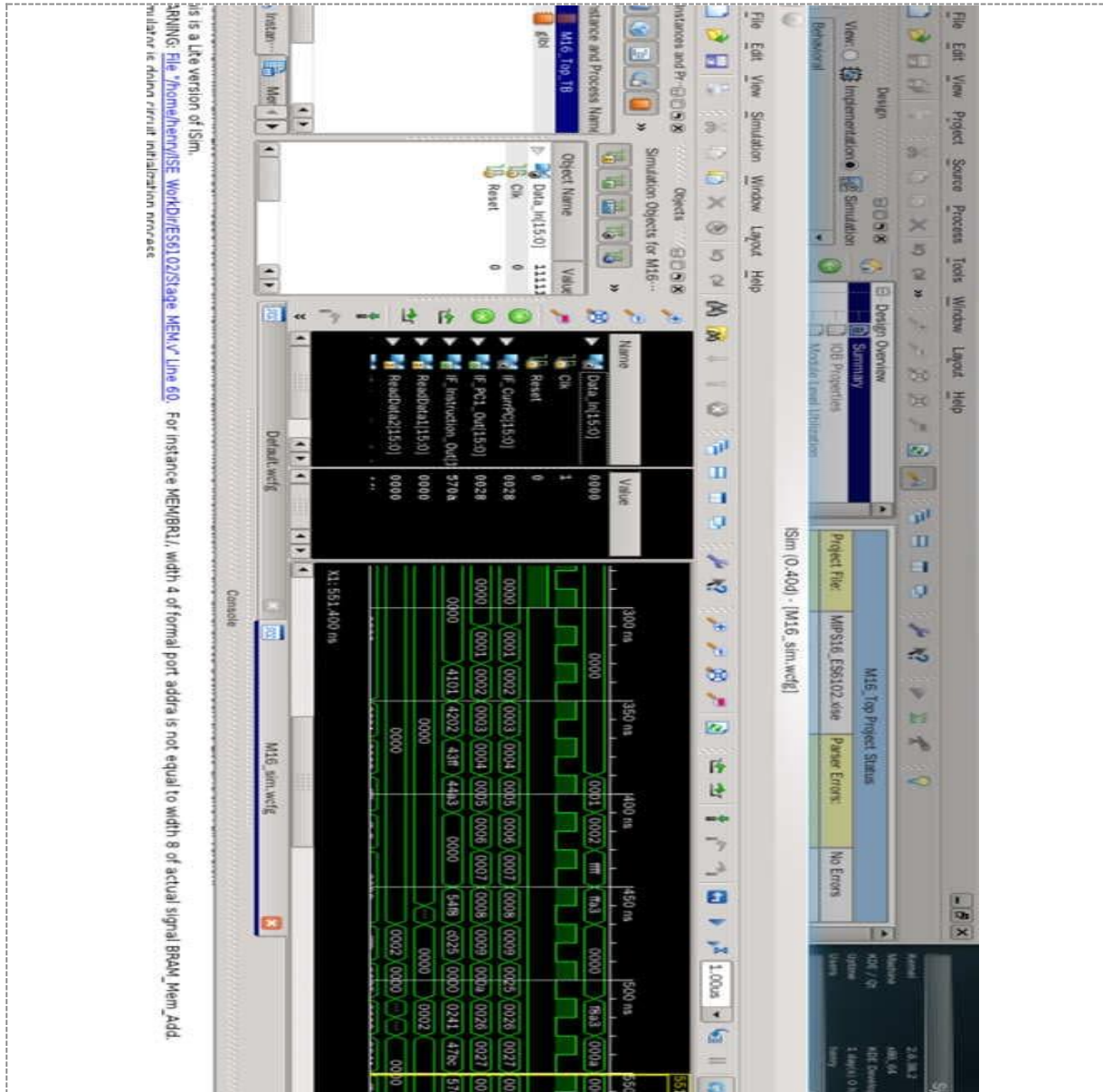


Figure 9: Vivado FPGA design environment

5.1 Vivado Design Flow

Design flow: Project Creation → HDL Design → Constraints → Simulation → Synthesis → Implementation → Bitstream → Hardware Programming → Verification.

Stage	Purpose
Project Creation	Configure the FPGA project and target device
HDL Design	Describe the digital logic using Verilog/VHDL
Simulation	Verify logical behaviour before hardware mapping
Synthesis	Convert RTL into a hardware-oriented representation
Implementation	Map and route the design onto FPGA resources
Bitstream Generation	Create the configuration data for the FPGA
Hardware Programming	Configure the physical FPGA device
Verification	Check the actual hardware behaviour

HDL

Verilog and VHDL are the two common Hardware Description Languages used to describe digital logic for simulation and synthesis.

Simulation vs Hardware Testing

Simulation verifies logical correctness; synthesis and implementation prepare the design for actual FPGA hardware. Physical testing remains necessary to confirm real-world timing and electrical behaviour.

Advantages

- Supports the complete FPGA design cycle within one environment.
- Enables early detection of logical errors through simulation.
- Provides detailed reports on timing, resource utilisation, and design constraints.

Limitations

- Requires familiarity with HDL concepts.
- Synthesis and implementation runtimes can be significant for large designs.
- Effective use requires understanding of digital-design fundamentals.

5.2 Practical Demonstration Using Vivado

Demonstration: Simple LED Blinking / Digital Logic Design on FPGA (specific board to be selected based on laboratory availability).

Aim

To design and implement a simple digital logic circuit on an FPGA and verify its behaviour.

Design Requirements

Requirement	Description
FPGA Board	As specified by the laboratory/college
HDL Source	Verilog or VHDL description of the logic
Constraints File	Pin mapping and timing constraints

HDL Design

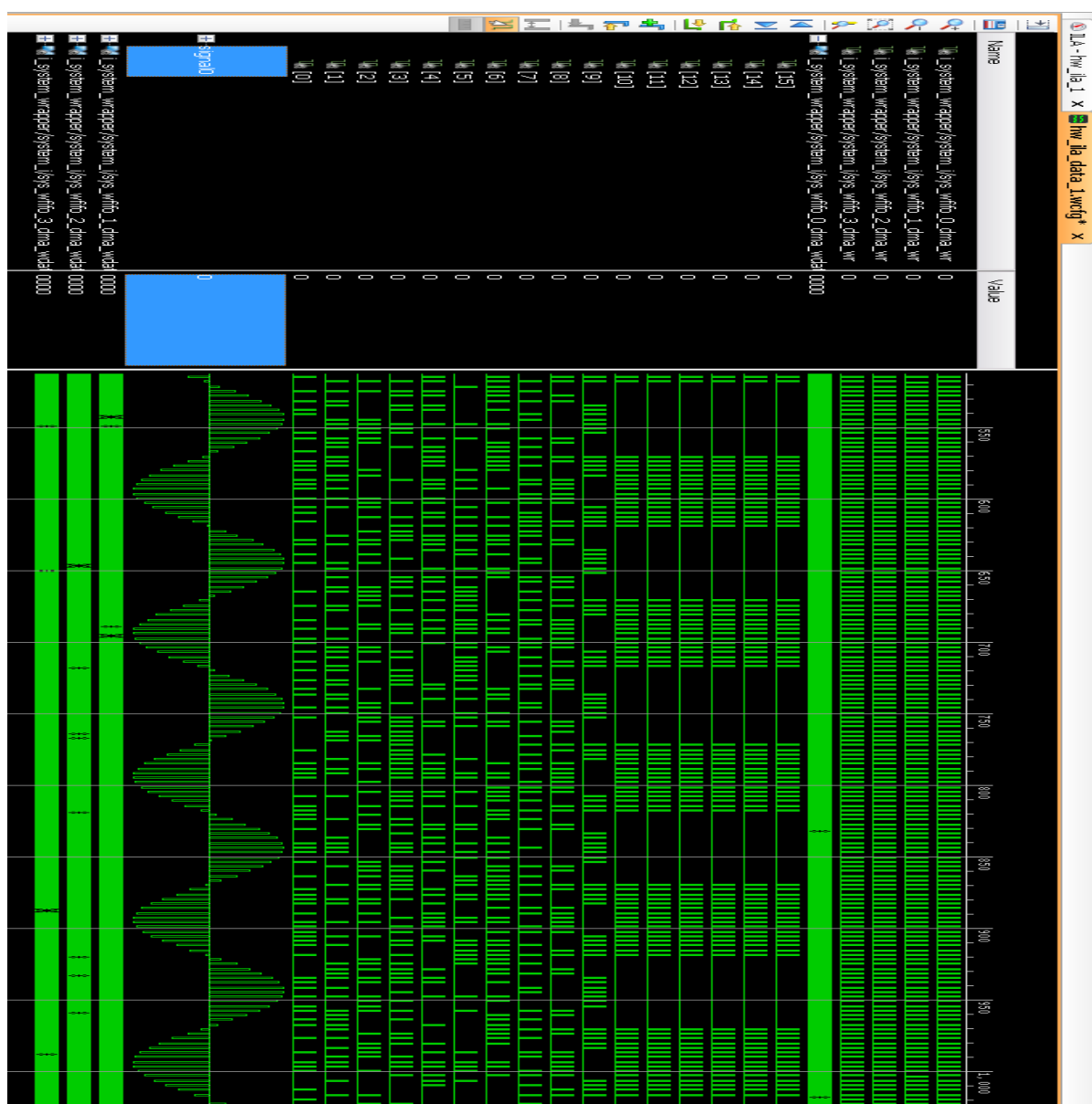
The HDL source describes the intended logic, such as a counter driving an LED at a visible blink rate, using standard Verilog/VHDL constructs.

Procedure

28. Create a new Vivado project.
29. Select the appropriate FPGA device or board.
30. Create the HDL source describing the digital logic.
31. Add the required constraints file.
32. Run behavioural simulation to verify logic.
33. Run synthesis.
34. Run implementation.
35. Generate the bitstream.
36. Program the FPGA hardware, where available.
37. Observe and verify the output.

Expected Result

The design should produce the intended output, such as a periodically blinking LED, confirming the flow from HDL description to hardware configuration.



6. Comparison of Multisim, Proteus and Vivado

Parameter	Multisim	Proteus	Vivado
Primary purpose	Circuit simulation	Circuit & embedded simulation	FPGA/digital design
Main domain	Analog/digital circuits	Embedded systems	Digital hardware/FPGA
Simulation focus	Component-level SPICE	Circuit + firmware	RTL/HDL logic
Hardware focus	Low	Moderate	High
Typical users	Electronics students	Embedded learners/engineers	FPGA/digital engineers
Design workflow	Schematic → simulate	Schematic → firmware → simulate	HDL → synthesis → implementation
Industry relevance	Circuit prototyping	Embedded prototyping	FPGA/ASIC-related design

The tools are not ranked against one another: Multisim suits component-level circuit analysis, Proteus extends this to embedded/microcontroller behaviour, and Vivado targets FPGA and digital hardware design.

When to Use Each Tool

- Multisim — suitable for circuit-level simulation and analysis.
- Proteus — useful for circuit and embedded-system simulation together.
- Vivado — appropriate for FPGA and digital hardware design workflows.

7. Industry Applications and Effective Use

Engineering software tools support multiple stages of the product-development cycle, including:

- Product development
- Prototyping
- Circuit verification
- Embedded-system development
- FPGA development
- Hardware debugging
- Design documentation
- Education and training
- Research and development

Simulation in the Engineering Development Cycle

Development cycle: Requirement → Design → Simulation → Verification → Prototype → Testing → Improvement → Final Implementation.

Effective Use of Engineering Software

- Understand the underlying engineering theory before relying on simulation.
- Select appropriate models and components for the design.
- Check units and parameters carefully.
- Verify simulation assumptions before drawing conclusions.
- Compare simulated results with theoretical calculations.
- Document the simulation setup for future reference.
- Perform hardware testing where required.
- Avoid blindly trusting simulation output without verification.
- Maintain software and component libraries appropriately.
- Follow applicable licensing requirements.

Simulation Limitations

- Component tolerances in real hardware.
- Environmental conditions such as temperature and humidity.
- Parasitic effects not fully captured in models.
- Manufacturing variations.
- Simplifications inherent in simulation models.

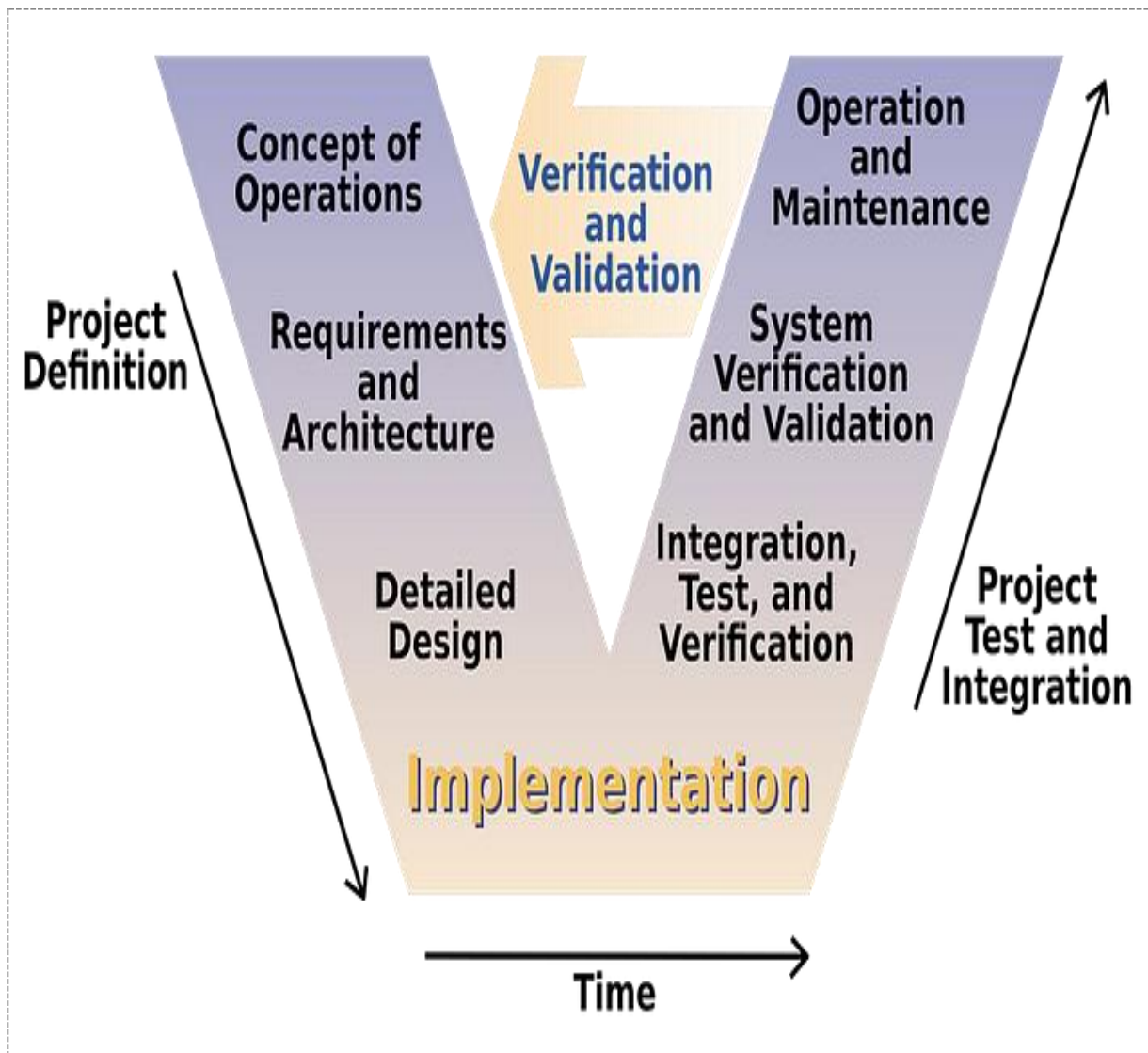


Figure 14: Typical engineering design and verification cycle

8. Skills Developed Through Engineering Software

Working with tools such as Multisim, Proteus, and Vivado helps students build practical skills relevant to the E&CE industry, including:

- Circuit analysis
- Digital electronics
- Embedded systems
- FPGA design
- HDL programming
- Simulation and verification
- Debugging
- Technical documentation
- Problem solving

Student Learning Path

Typical progression: Engineering Fundamentals → Basic Simulation → Circuit/Digital Design → Embedded/FPGA Projects → Hardware Verification → Documentation.

Industry Relevance

These skills relate directly to electronics design, embedded systems, FPGA development, and product development, strengthening a student's technical profile (without implying guaranteed employment or salary outcomes).

Recommended Learning Approach

38. Learn engineering fundamentals thoroughly.
39. Learn one simulation tool in depth.
40. Build small, well-documented projects.
41. Compare simulation results with theoretical calculations.
42. Progress gradually to embedded or FPGA tools.
43. Test designs on physical hardware when possible.
44. Document projects in a professional manner.

9. Conclusion

Software tools play an increasingly important role in modern E&CE, supporting design, simulation, verification, and implementation. Multisim supports circuit-level simulation, Proteus extends this to embedded/microcontroller behaviour, and Vivado addresses FPGA and digital hardware design.

Simulation before physical implementation reduces design iterations and catches errors early, but results should always be interpreted alongside sound theoretical understanding, since no simulation perfectly replicates real hardware. Overall, familiarity with these tools equips students with practical, industry-relevant skills that complement their coursework.

10. References

- National Instruments — Multisim official product documentation.
- Labcenter Electronics — Proteus Design Suite official documentation.
- AMD (Xilinx) — Vivado Design Suite official documentation.
- Savitribai Phule Pune University — Electronics and Computer Engineering curriculum reference material.
- Standard undergraduate textbooks on electronic circuits, embedded systems, and digital design (to be cited specifically as per college guidelines).

Note: References should be verified and expanded with exact citation details (author, edition, year, publisher/URL) as required by college submission guidelines before final submission.